

Fiche de révision - Python pour le bac de mathématiques

- ▶ Python apparaît surtout dans des questions sur les suites et éventuellement les exercices sur les fonctions.

- ▶ Python apparaît surtout dans des questions sur les suites et éventuellement les exercices sur les fonctions.
- ▶ À savoir lire : affectation, test, boucle `for`, boucle `while`, fonction, `return`.

- ▶ Python apparaît surtout dans des questions sur les suites et éventuellement les exercices sur les fonctions.
- ▶ À savoir lire : affectation, test, boucle `for`, boucle `while`, fonction, `return`.
- ▶ Savoir compléter une ligne de code ou interpréter le résultat donné par un algorithme.

Calculer un terme u_n

```
def suite(n):                                # renvoie  $u_n$ 
```

$$u_0 = 3 \text{ et } u_{n+1} = \frac{4}{5 - u_n}$$

Calculer un terme u_n

```
def suite(n):  
    u = 3
```

renvoie u_n
u_0

$$u_0 = 3$$

Calculer un terme u_n

```
def suite(n):  
    u = 3  
    for i in range(n):  
        # renvoie  $u_n$   
        #  $u_0$   
        #  $n$  tours
```

n répétitions de la récurrence

Calculer un terme u_n

```
def suite(n):  
    u = 3  
    for i in range(n):  
        u = 4/(5-u)
```

renvoie u_n
u_0
n tours
récurrence

$$u_{i+1} = \frac{4}{5 - u_i}$$

Calculer un terme u_n

```
def suite(n):  
    u = 3  
    for i in range(n):  
        u = 4/(5-u)  
    return u
```

renvoie u_n
u_0
n tours
récurrence
renvoie u_n

valeur renvoyée : u_n

```
def seuil():                                     # cherche un rang
```

$u_0 = 1$; on cherche le premier n tel que $u_n > 14$

Trouver un seuil

```
def seuil():                                # cherche un rang  
    n = 0                                   # rang
```

rang courant : n

Trouver un seuil

```
def seuil():  
    n = 0  
    u = 1  
# cherche un rang  
# rang  
#  $u_0$ 
```

$$u_0 = 1$$

Trouver un seuil

```
def seuil():                                # cherche un rang
    n = 0                                    # rang
    u = 1                                    #  $u_0$ 
    while u <= 14:                           # condition
```

on continue tant que $u_n \leq 14$

Trouver un seuil

```
def seuil():                                # cherche un rang
    n = 0                                    # rang
    u = 1                                    #  $u_0$ 
    while u <= 14:                            # condition
        u = -0.02*u*u + 1.3*u                # récurrence
```

$$u_{n+1} = -0,02u_n^2 + 1,3u_n$$

Trouver un seuil

```
def seuil():  
    n = 0  
    u = 1  
    while u <= 14:  
        u = -0.02*u*u + 1.3*u  
        n = n + 1
```

cherche un rang
rang
u_0
condition
récurrence
rang suivant

on passe au rang suivant

Trouver un seuil

```
def seuil():                                # cherche un rang
    n = 0                                    # rang
    u = 1                                    #  $u_0$ 
    while u <= 14:                            # condition
        u = -0.02*u*u + 1.3*u                # récurrence
        n = n + 1                            # rang suivant
    return n                                  # rang renvoyé
```

premier rang où $u_n > 14$

Obtenir un rang et une valeur

```
def algo(p):
```

précision p

$$u_0 = 2 \text{ et } u_{n+1} = \frac{2u_n + 1}{u_n + 2}$$

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2
```

précision p
u_0

$$u_0 = 2$$

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2  
    n = 0
```

précision p
u_0
rang

rang courant : n

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2  
    n = 0  
    while u - 1 > p:
```

précision p
u_0
rang
condition

on continue tant que $u_n - 1 > p$

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2  
    n = 0  
    while u - 1 > p:  
        u = (2*u + 1)/(u + 2)
```

précision p
u_0
rang
condition
récurrence

$$u_{n+1} = \frac{2u_n + 1}{u_n + 2}$$

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2  
    n = 0  
    while u - 1 > p:  
        u = (2*u + 1)/(u + 2)  
        n = n + 1
```

précision p
u_0
rang
condition
récurrence
rang suivant

on passe au rang suivant

Obtenir un rang et une valeur

```
def algo(p):  
    u = 2  
    n = 0  
    while u - 1 > p:  
        u = (2*u + 1)/(u + 2)  
        n = n + 1  
    return (n, u)
```

précision p
u_0
rang
condition
récurrence
rang suivant
renvoie (n, u_n)

valeurs renvoyées : (n, u_n)

```
def racine(a, b):
```

```
# intervalle
```

on travaille sur l'intervalle $[a, b]$


```
def racine(a, b):  
    while abs(b-a) >= 0.001:
```

intervalle

condition

on s'arrête quand la largeur est $< 10^{-3}$

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2
```

intervalle
condition
milieu

milieu : $m = \frac{a + b}{2}$

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:
```

intervalle
condition
milieu
test

on regarde le signe de $f(m)$

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m
```

intervalle
condition
milieu
test
borne gauche

on garde la moitié droite

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:
```

intervalle
condition
milieu
test
borne gauche
sinon

sinon : on garde l'autre moitié

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m
```

intervalle
condition
milieu
test
borne gauche
sinon
borne droite

on garde la moitié gauche

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m  
    return m
```

intervalle
condition
milieu
test
borne gauche
sinon
borne droite
valeur approchée

valeur approchée de la racine

- ▶ Mettre la condition du `while` dans le mauvais sens.

Erreurs classiques à éviter

- ▶ Mettre la condition du `while` dans le mauvais sens.
- ▶ Oublier une mise à jour : `u = ...` ou `n = n + 1`.

Erreurs classiques à éviter

- ▶ Mettre la condition du `while` dans le mauvais sens.
- ▶ Oublier une mise à jour : `u = ...` ou `n = n + 1`.
- ▶ Placer `return` dans la boucle alors qu'il doit souvent être après la boucle.

Erreurs classiques à éviter

- ▶ Mettre la condition du `while` dans le mauvais sens.
- ▶ Oublier une mise à jour : `u = ...` ou `n = n + 1`.
- ▶ Placer `return` dans la boucle alors qu'il doit souvent être après la boucle.
- ▶ Confondre le rang renvoyé et la valeur du terme.

Exercice 1 – Énoncé

On considère la suite (u_n) définie par $u_0 = 4$ et $u_{n+1} = 0,8u_n + 1,2$.

```
def terme(n):  
    u = 4  
    for i in range(n):  
        u = 0.8*u + 1.2  
    return u
```

Que renvoie `terme(0)` ? Que calcule la fonction `terme` ?

Exercice 1 – Réponse

- Pour `terme(0)`, la boucle `for i in range(0)` ne s'exécute pas.

Exercice 1 – Réponse

- ▶ Pour `terme(0)`, la boucle `for i in range(0)` ne s'exécute pas.
- ▶ La variable `u` reste égale à 4.

Exercice 1 – Réponse

- Pour `terme(0)`, la boucle `for i in range(0)` ne s'exécute pas.
- La variable `u` reste égale à 4.

`terme(0)` renvoie 4.

Exercice 1 – Réponse

- Pour `terme(0)`, la boucle `for i in range(0)` ne s'exécute pas.
- La variable `u` reste égale à 4.

`terme(0)` renvoie 4.

- Après n passages dans la boucle, la variable `u` contient u_n .

Exercice 1 – Réponse

- Pour `terme(0)`, la boucle `for i in range(0)` ne s'exécute pas.
- La variable `u` reste égale à 4.

`terme(0)` renvoie 4.

- Après n passages dans la boucle, la variable `u` contient u_n .

La fonction `terme(n)` renvoie u_n .

Exercice 2 – Énoncé

Pour $u_0 = 1$ et $u_{n+1} = 1,5u_n + 2$, compléter la fonction qui renvoie u_n .

```
def suite(n):  
    u = ...  
    for i in range(...):  
        u = ...  
    return ...
```

Exercice 2 – Réponse

```
def suite(n):
```

```
# renvoie  $u_n$ 
```

Exercice 2 – Réponse

```
def suite(n):  
    u = 1
```

```
# renvoie  $u_n$ 
```

```
#  $u_0$ 
```

Exercice 2 – Réponse

```
def suite(n):  
    u = 1  
    for i in range(n):
```

renvoie u_n
u_0
n tours

Exercice 2 – Réponse

```
def suite(n):  
    u = 1  
    for i in range(n):  
        u = 1.5*u + 2
```

renvoie u_n
u_0
n tours
récurrence

Exercice 2 – Réponse

```
def suite(n):  
    u = 1  
    for i in range(n):  
        u = 1.5*u + 2  
    return u
```

renvoie u_n
u_0
n tours
récurrence
renvoie u_n

Exercice 2 – Réponse

```
def suite(n):  
    u = 1  
    for i in range(n):  
        u = 1.5*u + 2  
    return u
```

renvoie u_n
u_0
n tours
récurrence
renvoie u_n

La fonction renvoie u_n .

Exercice 3 – Énoncé

```
def algo():  
    n = 0  
    u = 3  
    while u < 20:  
        u = 1.1*u + 2  
        n = n + 1  
    return (n, u)
```

Interpréter précisément les deux nombres renvoyés.

Exercice 3 – Réponse

- La boucle s'arrête dès que $u \geq 20$.

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83 \quad u_3 = 10,613$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83 \quad u_3 = 10,613 \quad u_4 = 13,6743$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83 \quad u_3 = 10,613 \quad u_4 = 13,6743 \quad u_5 = 17,04173$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83 \quad u_3 = 10,613 \quad u_4 = 13,6743 \quad u_5 = 17,04173 \quad u_6 = 20,745903$$

Exercice 3 – Réponse

- ▶ La boucle s'arrête dès que $u \geq 20$.
- ▶ Le script renvoie le rang atteint puis la valeur correspondante.

Il renvoie le premier rang n tel que $u_n \geq 20$, puis u_n .

$$u_0 = 3 \quad u_1 = 5,3 \quad u_2 = 7,83 \quad u_3 = 10,613 \quad u_4 = 13,6743 \quad u_5 = 17,04173 \quad u_6 = 20,745903$$

Le script renvoie (6, 20,745903).

Exercice 4 – Énoncé

Pour $u_0 = 5$ et $u_{n+1} = 0,9u_n + 3$, compléter le script qui renvoie le plus petit entier n tel que $u_n > 20$.

```
def seuil():  
    n = 0  
    u = 5  
    while ..... :  
        .....  
        .....  
    return n
```

Exercice 4 – Réponse

```
def seuil():
```

```
# cherche un seuil
```

Exercice 4 – Réponse

```
def seuil():  
    n = 0
```

```
# cherche un seuil  
# rang
```

Exercice 4 – Réponse

```
def seuil():
```

```
    n = 0
```

```
    u = 5
```

```
# cherche un seuil
```

```
# rang
```

```
#  $u_0$ 
```


Exercice 4 – Réponse

```
def seuil():  
    n = 0  
    u = 5  
    while u <= 20:
```

cherche un seuil
rang
u_0
tant que $u \leq 20$

Exercice 4 – Réponse

```
def seuil():  
    n = 0  
    u = 5  
    while u <= 20:  
        u = 0.9*u + 3
```

cherche un seuil
rang
u_0
tant que $u \leq 20$
récurrence

Exercice 4 – Réponse

```
def seuil():  
    n = 0  
    u = 5  
    while u <= 20:  
        u = 0.9*u + 3  
        n = n + 1
```

cherche un seuil
rang
u_0
tant que $u \leq 20$
récurrence
rang suivant

Exercice 4 – Réponse

```
def seuil():  
    n = 0  
    u = 5  
    while u <= 20:  
        u = 0.9*u + 3  
        n = n + 1  
    return n
```

cherche un seuil
rang
u_0
tant que $u \leq 20$
récurrence
rang suivant
rang renvoyé

Exercice 4 – Réponse

```
def seuil():  
    n = 0  
    u = 5  
    while u <= 20:  
        u = 0.9*u + 3  
        n = n + 1  
    return n
```

cherche un seuil
rang
u_0
tant que $u \leq 20$
récurrence
rang suivant
rang renvoyé

Le premier rang obtenu est $n = 9$.

Exercice 5 – Énoncé

On cherche le plus petit entier n tel que $u_n < 0,1$, avec $u_0 = 2$ et $u_{n+1} = 0,4u_n$. Quelle condition convient ?

- ▶ A. `while u < 0.1:`
- ▶ B. `while u <= 0.1:`
- ▶ C. `while u >= 0.1:`
- ▶ D. `while n >= 0.1:`

Exercice 5 – Réponse

- ▶ On veut s'arrêter quand $u < 0,1$.

Exercice 5 – Réponse

- ▶ On veut s'arrêter quand $u < 0,1$.
- ▶ Donc on continue tant que $u \geq 0,1$.

Exercice 5 – Réponse

- ▶ On veut s'arrêter quand $u < 0,1$.
- ▶ Donc on continue tant que $u \geq 0,1$.

Réponse C : `while u >= 0.1:`

Exercice 6 – Énoncé

Pour $u_0 = 5$, $u_{n+1} = \sqrt{u_n + 1}$ et $\ell = \frac{1 + \sqrt{5}}{2}$, interpréter `seuil(3)`.

```
from math import sqrt
def seuil(p):
    u = 5
    i = 0
    l = (1 + sqrt(5))/2
    while abs(u-l) >= 10**(-p):
        u = sqrt(u+1)
        i = i + 1
    return i
```

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

$$u_4 \approx 1,640$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

$$u_4 \approx 1,640 \quad u_5 \approx 1,625$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

$$u_4 \approx 1,640 \quad u_5 \approx 1,625 \quad u_6 \approx 1,620$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

$$u_4 \approx 1,640 \quad u_5 \approx 1,625 \quad u_6 \approx 1,620 \quad u_7 \approx 1,619$$

Exercice 6 – Réponse

- ▶ Avec $p = 3$, la condition est $|u - \ell| \geq 10^{-3}$.
- ▶ Le script renvoie le premier rang i tel que $|u_i - \ell| < 10^{-3}$.

seuil(3) renvoie le premier rang où l'écart à ℓ est strictement inférieur à 0,001.

$$u_0 = 5 \quad u_1 \approx 2,449 \quad u_2 \approx 1,857 \quad u_3 \approx 1,690$$

$$u_4 \approx 1,640 \quad u_5 \approx 1,625 \quad u_6 \approx 1,620 \quad u_7 \approx 1,619$$

Le script renvoie 7.

Exercice 7 – Énoncé

Pour $u_0 = 8$ et $u_{n+1} = u_n - \ln\left(\frac{u_n}{4}\right)$:

```
from math import log
def mystere(k):
    u = 8
    S = 0
    for i in range(k):
        S = S + u
        u = u - log(u/4)
    return S
```

Que représente `mystere(10)` ?

Exercice 7 – Réponse

- La boucle effectue 10 passages.

Exercice 7 – Réponse

- ▶ La boucle effectue 10 passages.
- ▶ À chaque passage, on ajoute le terme courant à S .

Exercice 7 – Réponse

- ▶ La boucle effectue 10 passages.
- ▶ À chaque passage, on ajoute le terme courant à S .
- ▶ Les termes ajoutés sont donc $u_0, u_1, \dots, u_9$.

Exercice 7 – Réponse

- ▶ La boucle effectue 10 passages.
- ▶ À chaque passage, on ajoute le terme courant à S .
- ▶ Les termes ajoutés sont donc $u_0, u_1, \dots, u_9$.

$$\text{mystere}(10) \text{ représente } u_0 + u_1 + \dots + u_9 = \sum_{i=0}^9 u_i.$$

Exercice 8 – Énoncé

Reprendre l'exercice précédent. Modifier la fonction pour qu'elle renvoie la moyenne des k premiers termes au lieu de leur somme.

Exercice 8 – Réponse

```
from math import log
```

```
# importe ln
```

Exercice 8 – Réponse

```
from math import log  
def mystere(k):
```

```
# importe ln  
# moyenne de  $k$  termes
```

Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
```

```
# importe ln
# moyenne de  $k$  termes
#  $u_0$ 
```

Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
    S = 0
```

```
# importe ln
# moyenne de  $k$  termes
#  $u_0$ 
# somme
```


Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
    S = 0
    for i in range(k):
```

importe ln
moyenne de k termes
u_0
somme
k tours

Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
    S = 0
    for i in range(k):
        S = S + u
```

```
# importe ln
# moyenne de  $k$  termes
#  $u_0$ 
# somme
#  $k$  tours
# ajoute  $u_i$ 
```

Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
    S = 0
    for i in range(k):
        S = S + u
        u = u - log(u/4)
```

importe ln
moyenne de k termes
u_0
somme
k tours
ajoute u_i
récurrence

Exercice 8 – Réponse

```
from math import log
def mystere(k):
    u = 8
    S = 0
    for i in range(k):
        S = S + u
        u = u - log(u/4)
    return S/k
```

importe ln
moyenne de k termes
u_0
somme
k tours
ajoute u_i
récurrence
moyenne

Exercice 8 – Réponse

```
from math import log                # importe ln
def mystere(k):                    # moyenne de k termes
    u = 8                          # u0
    S = 0                          # somme
    for i in range(k):             # k tours
        S = S + u                  # ajoute ui
        u = u - log(u/4)           # récurrence
    return S/k                     # moyenne
```

La fonction renvoie $\frac{u_0 + u_1 + \cdots + u_{k-1}}{k}$.

Exercice 9 – Énoncé

f est continue et strictement croissante sur $[1; 3]$; l'équation $f(x) = 0$ admet une unique solution α dans cet intervalle. Compléter cet algorithme de dichotomie.

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = ...  
        if f(m) < 0:  
            ...  
        else:  
            ...  
    return m
```

Exercice 9 – Réponse

```
def racine(a, b):                                # dichotomie
```

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:
```

dichotomie
largeur

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2
```

dichotomie
largeur
milieu

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:
```

dichotomie
largeur
milieu
test

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m
```

dichotomie
largeur
milieu
test
borne gauche

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m
```

dichotomie
largeur
milieu
test
borne gauche
sinon

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m
```

dichotomie
largeur
milieu
test
borne gauche
sinon
borne droite

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m  
    return m
```

dichotomie
largeur
milieu
test
borne gauche
sinon
borne droite
valeur approchée

Exercice 9 – Réponse

```
def racine(a, b):  
    while abs(b-a) >= 0.001:  
        m = (a+b)/2  
        if f(m) < 0:  
            a = m  
        else:  
            b = m  
    return m
```

dichotomie
largeur
milieu
test
borne gauche
sinon
borne droite
valeur approchée

La fonction renvoie une valeur approchée de α à 0,001 près.

Exercice 10 – Énoncé

Soit $f(x) = 4 \ln(1+x) - \frac{x^2}{25}$ sur $[2; 6,5]$. Interpréter les deux nombres renvoyés par `bornes(2)`.

```
from math import log
def bornes(n):
    p = 1/10**n
    x = 6
    while f(x)-x > 0:
        x = x + p
    return (x-p, x)
```


Exercice 10 – Réponse

► Avec $n = 2$, on a $p = 10^{-2} = 0,01$.

Exercice 10 – Réponse

- ▶ Avec $n = 2$, on a $p = 10^{-2} = 0,01$.
- ▶ Le script avance de 0,01 en 0,01 à partir de 6.

Exercice 10 – Réponse

- ▶ Avec $n = 2$, on a $p = 10^{-2} = 0,01$.
- ▶ Le script avance de 0,01 en 0,01 à partir de 6.
- ▶ Il s'arrête au premier x tel que $f(x) - x \leq 0$.

Exercice 10 – Réponse

- ▶ Avec $n = 2$, on a $p = 10^{-2} = 0,01$.
- ▶ Le script avance de 0,01 en 0,01 à partir de 6.
- ▶ Il s'arrête au premier x tel que $f(x) - x \leq 0$.

bornes(2) renvoie (6,36 ; 6,37).

Exercice 10 – Réponse

- ▶ Avec $n = 2$, on a $p = 10^{-2} = 0,01$.
- ▶ Le script avance de 0,01 en 0,01 à partir de 6.
- ▶ Il s'arrête au premier x tel que $f(x) - x \leq 0$.

bornes(2) renvoie (6,36 ; 6,37).

La solution de $f(x) = x$ est donc encadrée par $6,36 < \alpha < 6,37$.

Exercice 12 – Énoncé

Pour $u_0 = 0$ et $u_{n+1} = 5u_n - 8n + 6$, compléter la fonction qui renvoie u_n .

```
def suite_u(n):  
    u = ...  
    for i in range(...):  
        u = ...  
    return u
```

Exercice 12 – Réponse

```
def suite_u(n):                                     # renvoie  $u_n$ 
```

Exercice 12 – Réponse

```
def suite_u(n):  
    u = 0
```

```
# renvoie  $u_n$ 
```

```
#  $u_0$ 
```


Exercice 12 – Réponse

```
def suite_u(n):  
    u = 0  
    for i in range(n):
```

renvoie u_n
u_0
n tours

Exercice 12 – Réponse

```
def suite_u(n):  
    u = 0  
    for i in range(n):  
        u = 5*u - 8*i + 6
```

renvoie u_n
u_0
n tours
récurrence

Exercice 12 – Réponse

```
def suite_u(n):  
    u = 0  
    for i in range(n):  
        u = 5*u - 8*i + 6  
    return u
```

renvoie u_n
u_0
n tours
récurrence
renvoie u_n

Exercice 12 – Réponse

```
def suite_u(n):  
    u = 0  
    for i in range(n):  
        u = 5*u - 8*i + 6  
    return u
```

renvoie u_n
u_0
n tours
récurrence
renvoie u_n

La fonction renvoie u_n .

Exercice 13 – Énoncé

On considère la suite définie par $u_1 = 3$ et $u_{n+1} = 1,1u_n + 1,3$.

```
def seuil(p):  
    n = 1  
    u = 3  
    while u <= p:  
        n = n + 1  
        u = 1.1*u + 1.3  
    return n
```

Interpréter `seuil(8.5)`.

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3$$

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3 \quad u_2 = 4,6$$

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3 \quad u_2 = 4,6 \quad u_3 = 6,36$$

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3 \quad u_2 = 4,6 \quad u_3 = 6,36 \quad u_4 = 8,296$$

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3 \quad u_2 = 4,6 \quad u_3 = 6,36 \quad u_4 = 8,296 \quad u_5 = 10,4256$$

Exercice 13 – Réponse

- Le script renvoie le premier rang n tel que $u_n > 8,5$.

$$u_1 = 3 \quad u_2 = 4,6 \quad u_3 = 6,36 \quad u_4 = 8,296 \quad u_5 = 10,4256$$

`seuil(8.5)` renvoie 5.

Exercice 14 – Énoncé

Que renvoie ce script ?

```
def suite(k):  
    S = 0  
    for i in range(k):  
        S = S + (3/4)**k  
    return S
```

Exercice 14 – Réponse

- La boucle effectue k passages.

Exercice 14 – Réponse

- ▶ La boucle effectue k passages.
- ▶ À chaque passage, elle ajoute $\left(\frac{3}{4}\right)^k$.

Exercice 14 – Réponse

- La boucle effectue k passages.
- À chaque passage, elle ajoute $\left(\frac{3}{4}\right)^k$.

$$S = \underbrace{\left(\frac{3}{4}\right)^k + \cdots + \left(\frac{3}{4}\right)^k}_{k \text{ fois}}$$

Exercice 14 – Réponse

- La boucle effectue k passages.
- À chaque passage, elle ajoute $\left(\frac{3}{4}\right)^k$.

$$S = \underbrace{\left(\frac{3}{4}\right)^k + \cdots + \left(\frac{3}{4}\right)^k}_{k \text{ fois}} = k \left(\frac{3}{4}\right)^k$$

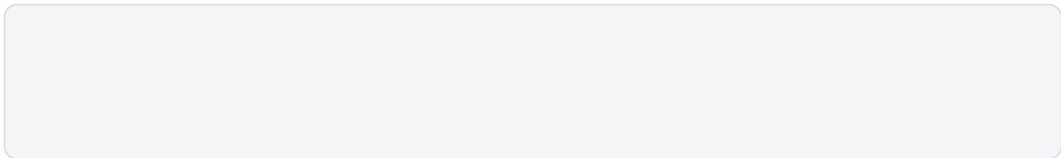
Exercice 14 – Réponse

- La boucle effectue k passages.
- À chaque passage, elle ajoute $\left(\frac{3}{4}\right)^k$.

$$S = \underbrace{\left(\frac{3}{4}\right)^k + \dots + \left(\frac{3}{4}\right)^k}_{k \text{ fois}} = k \left(\frac{3}{4}\right)^k$$

Le script renvoie $k \left(\frac{3}{4}\right)^k$.

Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.



Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.

Syntaxes utiles. `L = []` crée une liste vide ;

Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.

Syntaxes utiles. `L = []` crée une liste vide ;
`L.append(x)` ajoute une valeur ;

Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.

Syntaxes utiles. `L = []` crée une liste vide ;
`L.append(x)` ajoute une valeur ;
`L[0]` donne le premier élément ;

Les listes servent à mémoriser plusieurs valeurs au lieu de conserver seulement la dernière.

Syntaxes utiles. `L = []` crée une liste vide ;
`L.append(x)` ajoute une valeur ;
`L[0]` donne le premier élément ;
`len(L)` donne la longueur.

Construire la liste des premiers termes

```
from math import log
```

```
# importe ln
```

on utilise le logarithme népérien

Construire la liste des premiers termes

```
from math import log                                # importe ln  
def premiers_termes(k):                             # k termes
```

la fonction renvoie une liste de k termes

Construire la liste des premiers termes

```
from math import log                                # importe ln
def premiers_termes(k):                             # k termes
    L = []                                          # liste vide
```

on commence avec une liste vide

Construire la liste des premiers termes

```
from math import log                                # importe ln
def premiers_termes(k):                             # k termes
    L = []                                          # liste vide
    u = 5                                          #  $u_0$ 
```

$$u_0 = 5$$

Construire la liste des premiers termes

```
from math import log                                # importe ln
def premiers_termes(k):                             # k termes
    L = []                                          # liste vide
    u = 5                                          #  $u_0$ 
    for i in range(k):                            # k tours
```

k répétitions

Construire la liste des premiers termes

```
from math import log                                     # importe ln
def premiers_termes(k):                                  # k termes
    L = []                                               # liste vide
    u = 5                                                #  $u_0$ 
    for i in range(k):                                   # k tours
        L.append(u)                                       # ajoute  $u_i$ 
```

on ajoute le terme courant u_i

Construire la liste des premiers termes

```
from math import log                                # importe ln
def premiers_termes(k):                             # k termes
    L = []                                          # liste vide
    u = 5                                          # u0
    for i in range(k):                             # k tours
        L.append(u)                               # ajoute ui
        u = 2 + log(u*u - 3)                     # récurrence
```

$$u_{i+1} = 2 + \ln(u_i^2 - 3)$$

Construire la liste des premiers termes

```
from math import log
def premiers_termes(k):
    L = []
    u = 5
    for i in range(k):
        L.append(u)
        u = 2 + log(u*u - 3)
    return L
```

importe ln
k termes
liste vide
u_0
k tours
ajoute u_i
récurrence
renvoie la liste

la liste complète est renvoyée

Tester numériquement une propriété

```
def semble_croissante(L):                                # test
```

on teste si la liste semble croissante

Tester numériquement une propriété

```
def semble_croissante(L):  
    for i in range(len(L)-1):
```

test
indices

on parcourt les indices utiles

Tester numériquement une propriété

```
def semble_croissante(L):  
    for i in range(len(L)-1):  
        if L[i+1] < L[i]:  
            # test  
            # indices  
            # baisse ?
```

on vérifie si $L[i + 1] < L[i]$

Tester numériquement une propriété

```
def semble_croissante(L):  
    for i in range(len(L)-1):  
        if L[i+1] < L[i]:  
            return False
```

test
indices
baisse ?
pas croissante

une baisse suffit : la réponse est False

Tester numériquement une propriété

```
def semble_croissante(L):  
    for i in range(len(L)-1):  
        if L[i+1] < L[i]:  
            return False  
    return True
```

test
indices
baisse ?
pas croissante
croissante

aucune baisse trouvée : la réponse est True

À retenir. Dans ce type de script, u calcule le terme suivant et L mémorise les termes déjà obtenus.

À retenir. Dans ce type de script, u calcule le terme suivant et L mémorise les termes déjà obtenus.

Les indices de liste commencent à 0.

Mini-exercice – Énoncé

Que renvoie `termes(4)` ? Que représente la liste renvoyée ?

$$u_0 = 3$$

$$u_0 = 3 \quad u_1 = 0,5 \times 3 + 4 = 5,5$$

$$u_0 = 3 \quad u_1 = 0,5 \times 3 + 4 = 5,5 \quad u_2 = 0,5 \times 5,5 + 4 = 6,75$$

$$u_0 = 3 \quad u_1 = 0,5 \times 3 + 4 = 5,5 \quad u_2 = 0,5 \times 5,5 + 4 = 6,75 \quad u_3 = 0,5 \times 6,75 + 4 = 7,375$$

Mini-exercice – Réponse

$$u_0 = 3 \quad u_1 = 0,5 \times 3 + 4 = 5,5 \quad u_2 = 0,5 \times 5,5 + 4 = 6,75 \quad u_3 = 0,5 \times 6,75 + 4 = 7,375$$

termes(4) renvoie [3 ; 5,5 ; 6,75 ; 7,375].

$$u_0 = 3 \quad u_1 = 0,5 \times 3 + 4 = 5,5 \quad u_2 = 0,5 \times 5,5 + 4 = 6,75 \quad u_3 = 0,5 \times 6,75 + 4 = 7,375$$

`termes(4)` renvoie `[3 ; 5,5 ; 6,75 ; 7,375]`.

La liste contient les quatre premiers termes : u_0, u_1, u_2, u_3 .